hw_v3.py
Code Walkthrough

Harbor Watch real-time video analytics backend: architecture, modules, tensor dimensions, processing sequence, and issues found

Topic	Covered
Architecture	Front/rear camera feeds, threads, queues, CUDA streams
Models	RF-DETR (CCD), IR detector, classifiers, YOLOv26 fire/smoke, X3D conflict
Pipeline	Panorama, tiling, detection, ROI extraction, letterbox, classification
Review	Five issues observed in the implementation

Contents

1 What the code does
2 Overall architecture
3 Imports and environment
4 GPU device selection
5 Human conflict detection
6 Front and rear camera buffers
7 Socket.IO communication
8 Object detection models
9 Object classification models
10 Video acquisition (RTSPReader)
11 Queues and parallel processing
12 Panorama processing
13 Main GPU inference thread
14 Object detection
15 Detection filtering
16 ROI extraction
17 Letterbox resizing
18 Classification
19 Fire and smoke detection
20 Human conflict analytics thread
21 Video encoding
22 Sending results to the frontend
23 Complete GPU inference pipeline
24 Problems noticed in the code
25 Final summary

1. What the code does

hw_v3.py is the main backend pipeline for a real-time video analytics application. It receives video from two camera feeds, a **front** camera and a **rear** camera, processes it with GPU-accelerated operations, and runs multiple analytics models.

Main functionalities

Module	Purpose	Technology
Video acquisition	Read front and rear video feeds	PyAV, RTSP
Panorama processing	Combine and process camera images	PyTorch, CUDA
Object detection	Detect ships, boats and vessels	RF-DETR
Object classification	Classify detected objects	Custom classification model
ROI extraction	Extract detected objects from frames	PyTorch, ROI operations
Fire detection	Detect fire	YOLOv26
Smoke detection	Detect smoke	YOLOv26
Human conflict detection	Detect violent activity	X3D
Video streaming	Send processed images to the frontend	Socket.IO
Image encoding	Convert frames into JPEG	TurboJPEG

The application processes video continuously.

2. Overall architecture

This is the most important flowchart because it explains how the different components communicate.

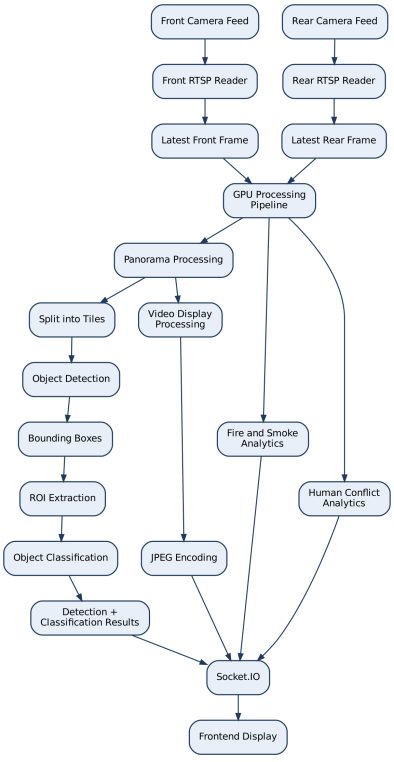


Figure 1. Overall system flowchart

The application follows these steps:

- Read video from the front and rear cameras.

- Store the latest frames.
- Transfer the frames to GPU processing.
- Resize and preprocess the images.
- Divide the panorama into smaller tiles.
- Run object detection.
- Extract the detected objects.
- Classify the detected objects.
- Generate analytics results.
- Encode the images.
- Send the results to the frontend.

Detection, classification and video streaming are different operations inside the same application. They do not need to execute sequentially: the code uses **CUDA streams, Python threads and queues** to run different parts of the pipeline concurrently.

3. Imports and environment

The file begins with:

```
import os, sys, torch
```

These are used for operating system operations, Python module management and GPU processing.

3.1 CUDA library loading

```
torch_dir = os.path.dirname(torch.__file__)
torch_lib = os.path.join(torch_dir, "lib")
os.add_dll_directory(torch_lib)
```

This adds the PyTorch library directory to the Windows DLL search path. The code also loads custom compiled backends:

```
backend1 = os.path.join(os.path.dirname(__file__), "fused_backend")
os.add_dll_directory(backend1)
sys.path.insert(0, backend1)
```

Similar entries exist for **build_roi** (backend2) and **scatter_labels** (backend3). These directories contain custom compiled operations:

```
import roi_fused
import fused_preprocess_new
import fused_preprocess_large
import split_concat
import scatter_fused
```

These are not ordinary Python functions. They are likely compiled CUDA or C++ extensions designed to accelerate image processing.

Operation	Purpose
`split_concat`	Split or combine image regions
`fused_preprocess_new`	Process images for one camera mode
`fused_preprocess_large`	Process images for another camera mode

Operation	Purpose
`roi_fused`	Accelerate ROI-related processing
`scatter_fused`	Accelerate label or result scattering

Combining multiple image-processing operations into a single GPU kernel can reduce kernel launch overhead, intermediate tensor creation, memory transfers and CPU processing overhead.

4. GPU device selection

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

This checks whether CUDA is available.

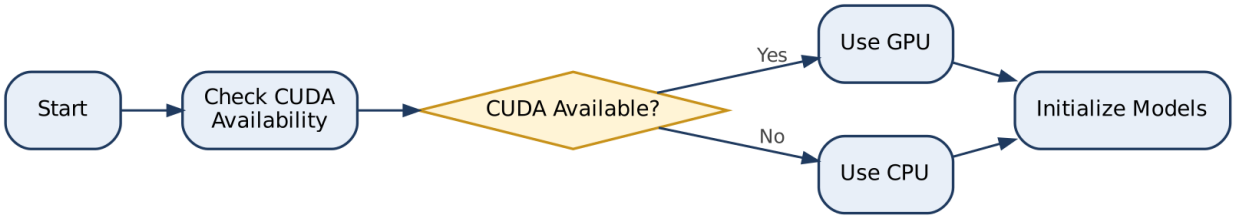


Figure 2. Device selection

Some parts of the application explicitly use `device="cuda:0"`, for example the human-conflict model. That model therefore assumes a CUDA-compatible GPU is available.

5. Human conflict detection

```
class HumanConflictDetector:
```

This class detects violent activity from a **sequence of video frames**. Unlike ordinary object detection it analyzes temporal information rather than a single image.

5.1 Model initialization

```
def __init__(self, model_path, device="cuda:0", clip_len=16,
             input_size=224, threshold=0.80):
```

Parameter	Value	Meaning
`clip_len`	16	Number of frames analyzed together
`input_size`	224	Spatial input resolution
`threshold`	0.80	Minimum confidence for a positive prediction
`device`	cuda:0	GPU used for inference

```
self.model = x3d.generate_model(
    x3d_version='M', n_classes=2, n_input_channels=3,
    task='class', dropout=0, base_bn_splits=1)
```

This creates an **X3D-M** video classification model that analyzes spatial and temporal information. It has two output classes, and the code assumes **0 = Violence**, **1 = Non-violence**. The inference code interprets the output accordingly.

5.2 Human-conflict prediction

`def predict(self, clip)` accepts a list of 16 frames (`[frame_1, ..., frame_16]`). Processing steps:

```
np_clip = np.stack(clip)
inputs  = torch.from_numpy(np_clip)
inputs  = inputs.to(self.device, non_blocking=True)
outputs = self.model(inputs)
probs   = self.softmax(outputs)
fight_prob = probs[0, 0].item()
pred       = outputs.argmax(1).item()
is_fight   = (pred == 0 and fight_prob >= self.threshold)
```

Tensor dimension flow

The input starts as **[T, H, W, C]**, for 16 frames **[16, H, W, 3]**, and is rearranged to **[1, 3, 16, 224, 224]**:

Dimension	Meaning
1	Batch size
3	RGB/BGR channels
16	Number of frames
224	Image height
224	Image width

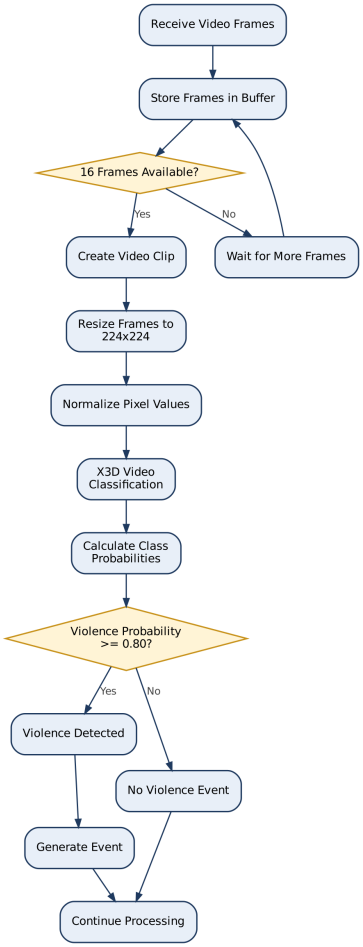


Figure 3. Human-conflict detection flow

5.3 Important observation

> The model reads `fight_prob = probs[0, 0].item()` and checks `pred == 0`, so class index 0 **must** correspond to violence. If your trained model uses the opposite class ordering, this code will produce incorrect results. The class mapping must match the training dataset.

6. Front and rear camera buffers

```
NUM_ZONES = 2
ZONE_W = 3840

FRONT_CONFLICT_BUFFERS = [
    deque(maxlen=human_conflict_detector.clip_len)
    for _ in range(NUM_ZONES)]
```

A matching `REAR_CONFLICT_BUFFERS` exists. This creates separate temporal buffers for the front and rear cameras, each storing up to 16 frames.

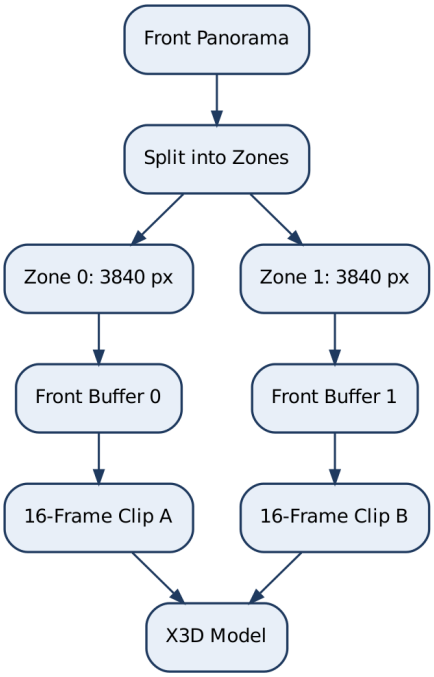


Figure 4. Buffer architecture

This lets the system analyze different spatial regions independently. For example, one zone might contain a group of people while another zone is empty.

7. Socket.IO communication

```
sio = socketio.Server(async_mode="eventlet", cors_allowed_origins="*")
app = socketio.WSGIApp(sio)
```

This creates the Socket.IO server. Two namespaces are used: `/front` and `/rear`, representing the front and rear camera interfaces.

7.1 Frontend controls

```
@sio.on("set_detection", namespace="/front")
def set_detection_front(sid, data):
    DETECTION_STATE["/front"] = bool(data.get("enabled", False))
```

This enables or disables detection for the front camera. `set_classification` controls classification and `set_event` controls analytics events.

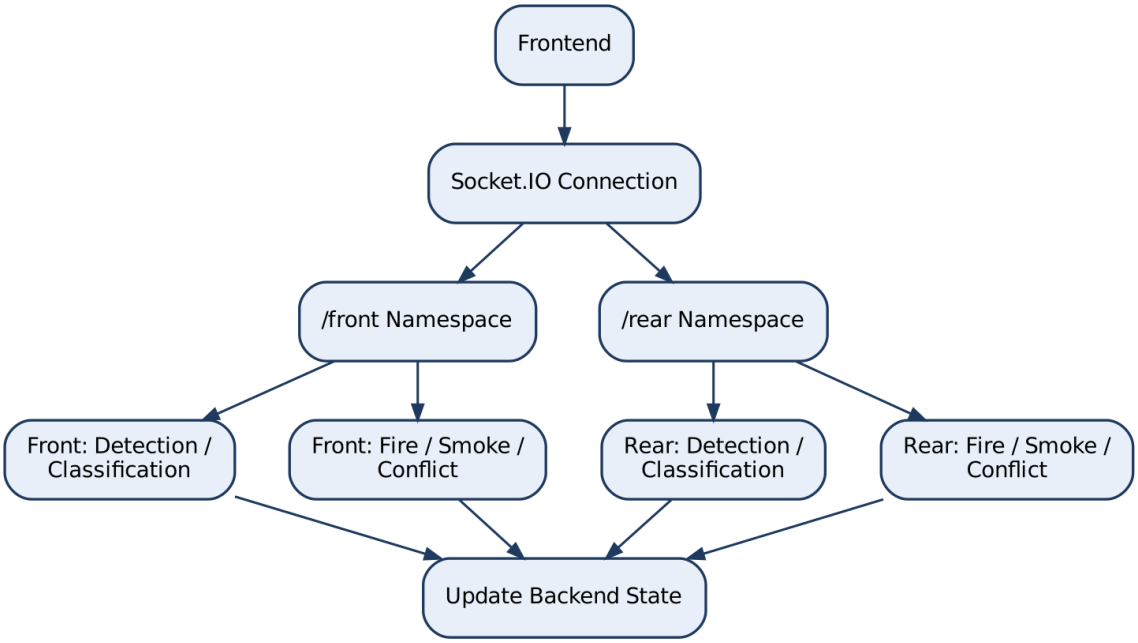


Figure 5. Socket.IO control flow

These controls are intended to determine which operations execute. However, there is an important issue in the main inference function, covered in Section 24.

8. Object detection models

Two detection models are initialized.

8.1 CCD detection model

```
detectionModelCCD = detectionModelRFDETRLarge(
    DETECTION_MODEL="models\ccdDetection\seaVessel_detection_CCD_RF_DETRLarge_rs_1120_updated_stats_b16/checkpoi
    DET_THRES=0.2)
```

Used for CCD camera mode. **DET_THRES = 0.2** is the configured detection threshold, which decides which predicted objects are accepted based on confidence.

8.2 IR detection model

```
detectionModelIR = detectionModel(
    CCD_DETECTION=r".\models\irDetection\seaVessel_detection_ir_ep_1000_rs_560\checkpoint_best_total.pth",
    DET_THRES=0.1)
```

Used for IR camera mode.

Camera mode	Detection model
CCD	RF-DETR Large
IR	Custom detection model

The model is selected from `sh.CAMERA_MODE` inside the main inference thread.

9. Object classification models

Two classification models are initialized.

```
classificationModelCCD = classificationModel(
    s.CCD_CLASSIFIER_MODEL_PATH, s.CCD_CLASSIFICATION,
    s.CLASSIFICATION_CLASSES_CCD, classes2int_dict,
    s.CLS_THRES_CCD, s.CLS_ACC_LIMIT_CCD)

classificationModelIR = classificationModel(
    s.IR_CLASSIFIER_MODEL_PATH, s.IR_CLASSIFICATION,
    s.CLASSIFICATION_CLASSES_IR, classes2int_dict,
    s.CLS_THRES_IR, s.CLS_ACC_LIMIT_IR)
```

These classify the objects found by the detectors. **Detection** answers *where is the object?* and **classification** answers *what type of object is it?* This is a two-stage architecture.

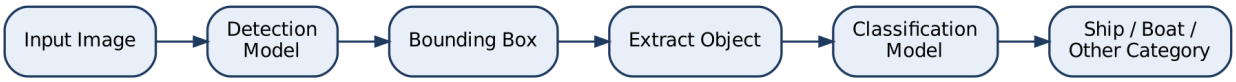


Figure 6. Two-stage detection and classification

10. Video acquisition (RTSPReader)

```
class RTSPReader:
```

This is one of the most important classes. It continuously reads video frames from a camera or video file.

```
def __init__(self, url, name="cam", reconnect_delay=2.0,
             out_w=8000, out_h=1080, source_mode=1, video_path=None):
```

Parameter	Purpose
`url`	RTSP camera URL
`name`	Camera identifier
`reconnect_delay`	Delay before reconnecting
`source_mode`	RTSP or offline video
`video_path`	Offline video path

Two input modes are supported via `SOURCE_MODE = 1`: **1 = RTSP live streams**, **0 = offline video files**.

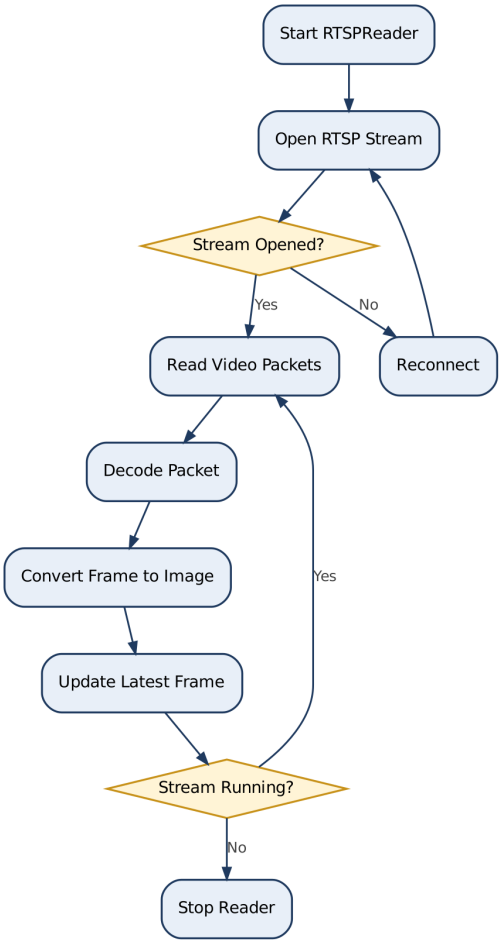


Figure 7. RTSP reader flow

10.1 Why a separate thread?

```
self._thread = threading.Thread(target=self._reader_loop, daemon=True,
                                name=f"reader-{name}")
```

Video decoding can be slow. If reading and inference shared one thread, the reader would stop while the model runs, causing video latency, frame buffering, stale frames and delayed analytics. A separate reader thread avoids this.

10.2 RTSP low-latency options

```
container = av.open(self.url, options={
    'rtsp_transport': 'tcp',
    'fflags': '+nobuffer+discardcorrupt',
    'flags': 'low_delay',
    'err_detect': 'ignore_err',
    'max_delay': '500000',
    'reorder_queue_size': '0',
    'stimeout': '5000000'}, timeout=15.0)
```

These options reduce latency and handle stream errors. For example, **+nobuffer+discardcorrupt** reduces buffering and discards corrupted packets. They do not guarantee low latency for the whole application; the processing pipeline and frontend can still add delay.

11. Queues and parallel processing

```
queue_1 = Queue(maxsize=1)
queue_2 = Queue(maxsize=1)
analytics_front = Queue(maxsize=1)
analytics_back = Queue(maxsize=1)
analytics_queue_fire_smoke = Queue(maxsize=2)
analytics_queue_human_conflict = Queue(maxsize=2)
```

Queues let threads exchange data. One thread reads video, another processes it, others run fire and human-conflict analytics, and another sends results to the frontend. Without queues these threads would have to coordinate directly.

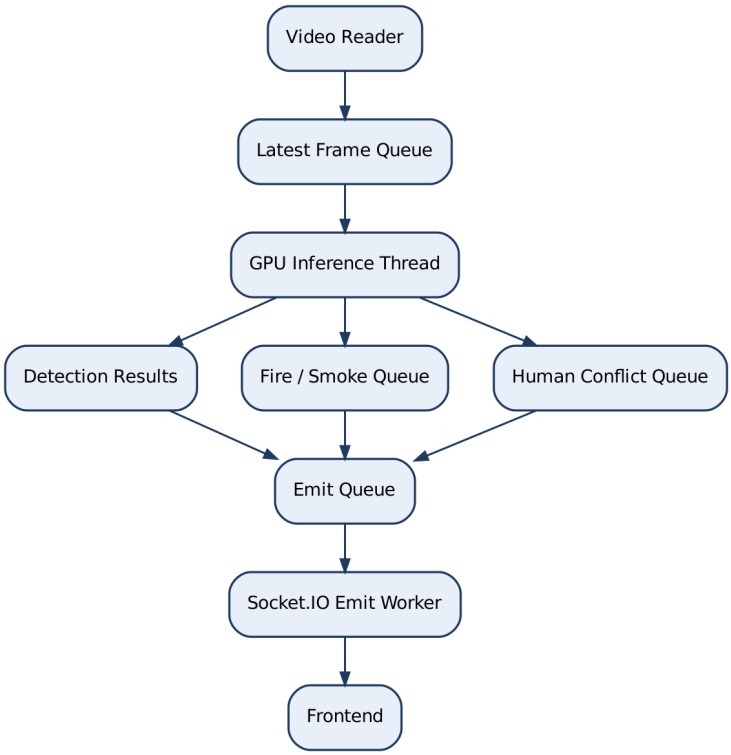


Figure 8. Queue architecture

> The analytics queues are small, e.g. **Queue(maxsize=2)**. When a queue is full, the code removes an older item and inserts a newer one. For real-time video, processing the latest frame is usually more valuable than processing an old one.

12. Panorama processing

This is the central image-processing stage. Front and rear frames are processed on the GPU using `split_concat.run()` and `pano_resize_encoding_optimize_new.run()`, preparing images for the detection and display pipelines. The code expects a wide **7680 × 1080** panorama and supports two camera modes.

Camera Mode 1

```
TILE_W = 1920
NUM_TILES = 4
```

The panorama is divided into four tiles, each 1920 × 1080.

Tile 1	Tile 2	Tile 3	Tile 4

Figure 9. 7680 × 1080 panorama split into 4 tiles

Camera Mode 0

```
TILE_W = 1120
NUM_TILES = 7
```

The panorama is divided into seven tiles, each 1120 × 1080.

T1	T2	T3	T4	T5	T6	T7

Figure 10. 7680 × 1080 panorama split into 7 tiles

Different preprocessing kernels are used for the two modes:

```
if sh.CAMERA_MODE == 1:
    tiles1 = fused_preprocess_large.run(...)
if sh.CAMERA_MODE == 0:
    tiles1 = fused_preprocess_new.run(...)
```

> The exact internal transformations of these compiled kernels cannot be established from hw_v3.py alone, because their implementations are in separately compiled backend modules.

13. Main GPU inference thread

```
def gpu_inference_thread():
```

This is the most important function in the file. It coordinates the whole GPU pipeline:

- Video acquisition
- GPU processing
- Panorama splitting
- Detection preprocessing
- Classification preprocessing
- Object detection
- ROI extraction
- Classification
- Result generation
- Video encoding and output

It uses multiple CUDA streams so GPU operations can run concurrently when dependencies permit:

```
stream_preprocess1
stream_preprocess2
stream_resize
stream_d2h
stream_tile_merge
stream_inference1
```

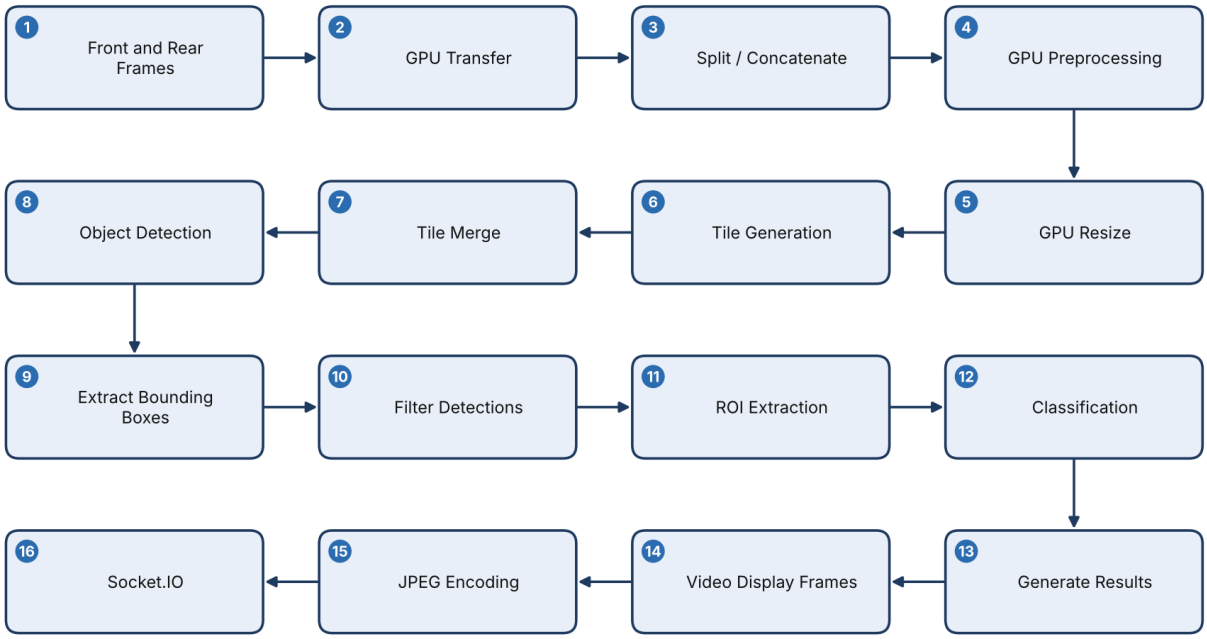


Figure 11. GPU pipeline (reads left to right, then right to left)

14. Object detection

```
if run_detection:
    with torch.cuda.stream(stream_inference1):
```

The detection model is selected by camera mode:

```
if sh.CAMERA_MODE == 1:
    num_classes, primaryModelOutputs = detectionModelCCD.predict(prev_tiles1)
elif sh.CAMERA_MODE == 0:
    num_classes, primaryModelOutputs = detectionModelIR.predict(prev_tiles11)
```

Boxes and confidences are then extracted:

```
boxes_list = [det.xyxy for det in primaryModelOutputs]
conf_list  = [det.confidence for det in primaryModelOutputs]
```

Bounding box format is **[x1, y1, x2, y2]**: x1 = left, y1 = top, x2 = right, y2 = bottom. Boxes are associated with their tiles because the detector processes multiple tiles in a batch.

15. Detection filtering

```
CLS_THRES = classificationModelCCD.CLS_THRES
mask = all_conf > CLS_THRES
d_roi_xyxy = all_boxes[mask][:MAX_BOX]
d_roi_tile = tile_ids[mask][:MAX_BOX]
```

The mask filters detections by confidence, and the maximum number of ROIs is **MAX_BOX = 256**.

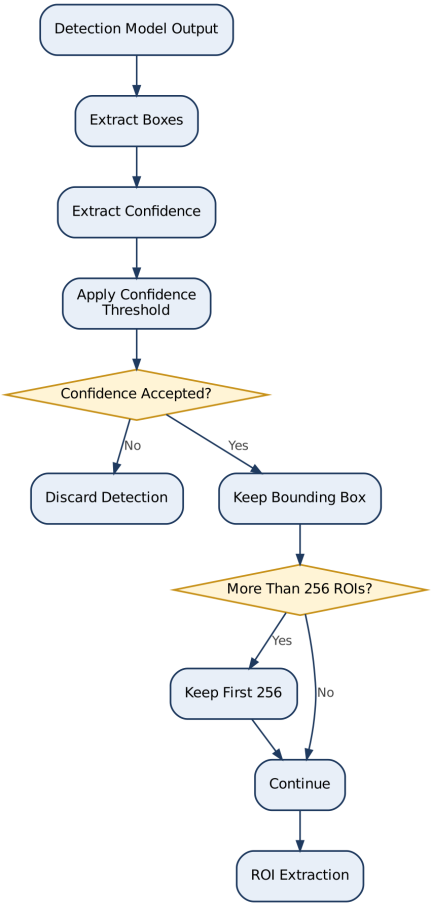


Figure 12. Detection filtering

> The threshold used here comes from the **classification** model configuration (classificationModelCCD.CLS_THRES) and is applied to the **detector's** confidence values. Detection confidence and classification confidence represent different things, so this is worth reviewing.

16. ROI extraction

ROI means Region of Interest. A bounding box identifies the region containing an object, and the code crops that region from the image:

```
crop = cls_tiles[tile_idx, :, y1:y2, x1:x2]
```

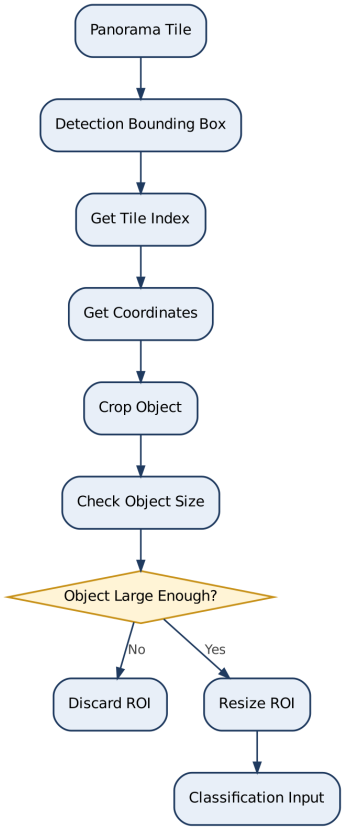


Figure 13. ROI extraction

The code also checks object size:

```
if (box_width <= s.CLASSIFICATION_WIDTH_LIMIT
        or box_height <= s.CLASSIFICATION_HEIGHT_LIMIT):
    print(f"Discarded detection {i}: object too small")
    continue
```

This prevents very small objects from reaching the classifier, since they may not contain enough visual information for reliable classification.

17. Letterbox resizing

```
def resize_with_letterbox(crop, final_size=384, pad_value=128):
```

This resizes the ROI to **384 × 384** while preserving aspect ratio. A 600 × 300 object has a 2:1 ratio; resizing straight to 384 × 384 would distort it. Instead the code scales proportionally and pads.



Figure 14. Letterbox flow

Step	Size
Original ROI	600 × 300
Resized ROI	384 × 192
Final image (with padding)	384 × 384

The padding value is **pad_value = 128**, creating gray borders. This suits classifiers that expect a fixed input size.

18. Classification

Once the ROI is extracted and resized, it goes to the classification model. Classification tiles are prepared first:

```
cls_tiles1 = (gpu_proc1.view(3, 1080, NUM_TILES, TILE_W)
                       .permute(2, 0, 1, 3).contiguous())
cls_tiles2 = (gpu_proc2.view(3, 1080, NUM_TILES, TILE_W)
                       .permute(2, 0, 1, 3).contiguous())
cls_tiles = torch.cat((cls_tiles1, cls_tiles2), dim=0)
```

The rear panorama is handled the same way, and the final concatenation combines front and rear tiles into one tensor.



Figure 15. Classification flow

The exact class names and postprocessing are determined by the configuration and classification module.

19. Fire and smoke detection

```
model_yolo_fire  = YOLO(r".\fire\YOLOv26_Fire-Smoke.pt").to(device=DEVICE)
model_yolo_smoke = YOLO(r".\fire\YOLOv26_Fire-Smoke.pt").to(device=DEVICE)

CLASS_MAP_FIRE_SMOKE = {"fire": 0, "smoke": 1}
CLASS_MAP_FIRE_SMOKE_model = {0: "fire", 1: "smoke"}
```

Both variables use the same weights. The analytics function `video_analytics_fire_smoke()` receives frames from `analytics_queue_fire_smoke` and processes them with `video_analytics_predict_dual_yolov26()`, imported from `fire_smoke_final_latest.infer`.

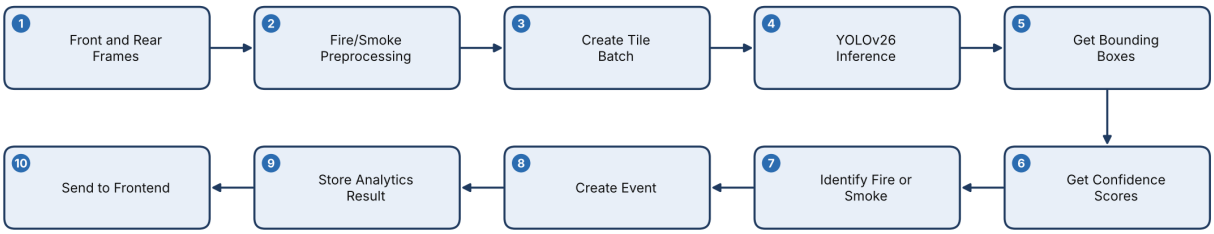


Figure 16. Fire and smoke flow

```
CONFIDENCE_THRESHOLD_FIRE  = 0.1
CONFIDENCE_THRESHOLD_SMOKE = 0.4
```

The fire threshold is lower than the smoke threshold.

20. Human conflict analytics thread

```
def video_analytics_human_conflict():
```

This receives frames from `analytics_queue_human_conflict`, divides the panorama into zones, and uses the temporal buffers from Section 6.

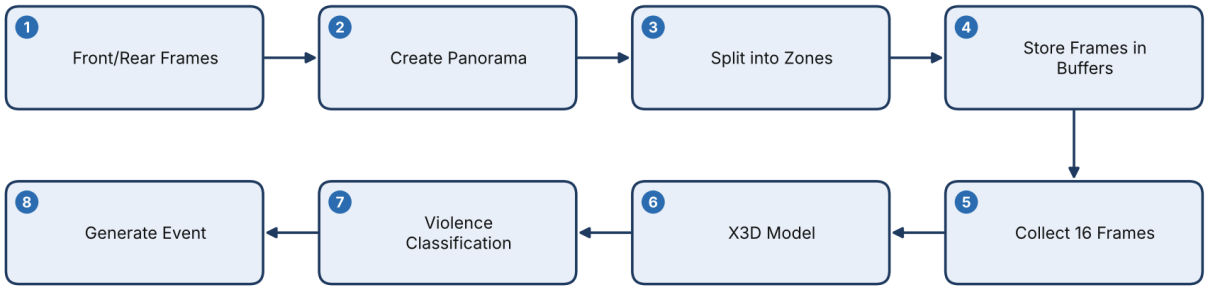


Figure 17. Human-conflict analytics sequence

Results are stored in `ANALYTICS_RESULT`. The shared dictionary is protected by `ANALYTICS_LOCK`, so multiple threads cannot modify it at the same time without synchronization.

21. Video encoding

```
def encode_frame(frame):
    buf = jpeg_turbo.encode(frame, quality=70)
    return base64.b64encode(buf).decode()
```

The frame is JPEG-encoded and the bytes are converted to Base64 text, so the frontend can receive the image through a Socket.IO message.



Figure 18. Encoding flow

quality = 70	Effect
Higher quality	Better image quality, larger encoded data
Lower quality	Smaller encoded data, lower image quality

22. Sending results to the frontend

```
emit_queue = Queue()

def emit_worker():
    event_name, data, frame_id = emit_queue.get()
    sio.emit(event_name, data)
```

The worker continuously reads messages from the queue and sends them through Socket.IO.

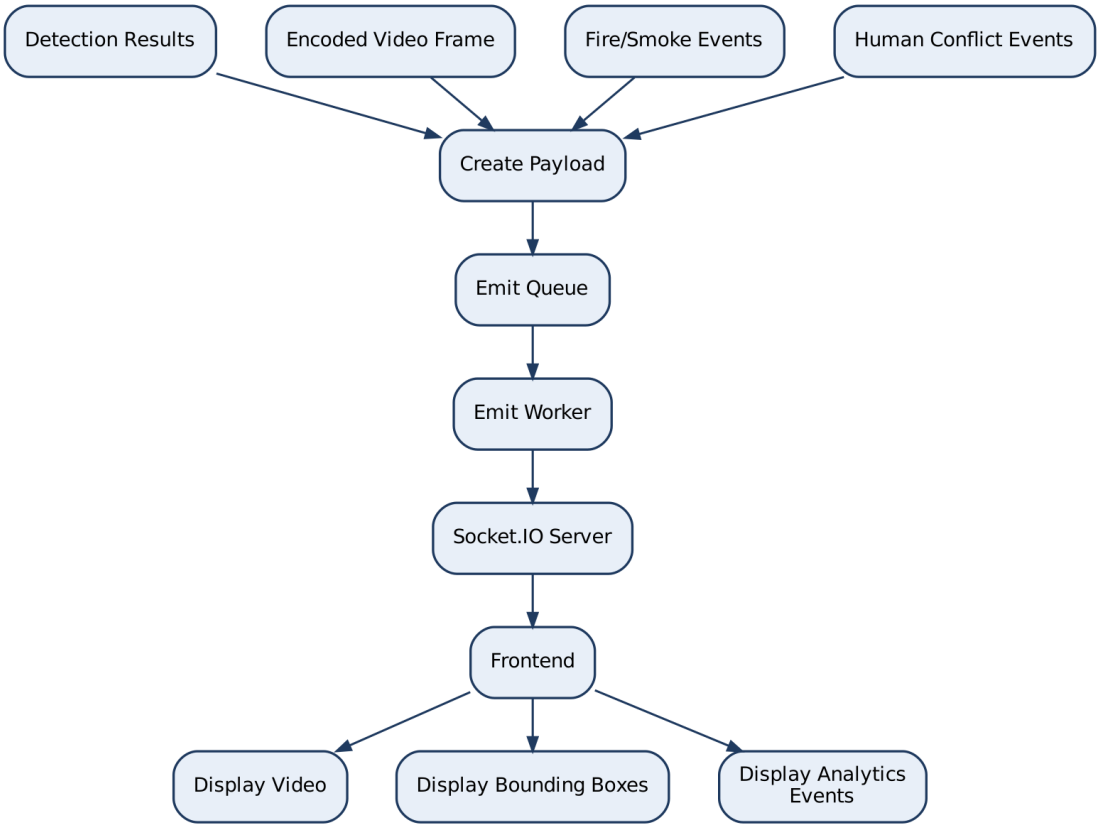


Figure 19. Final output flow

23. Complete GPU inference pipeline

This combines everything into one flowchart representing the main logic of gpu_inference_thread().

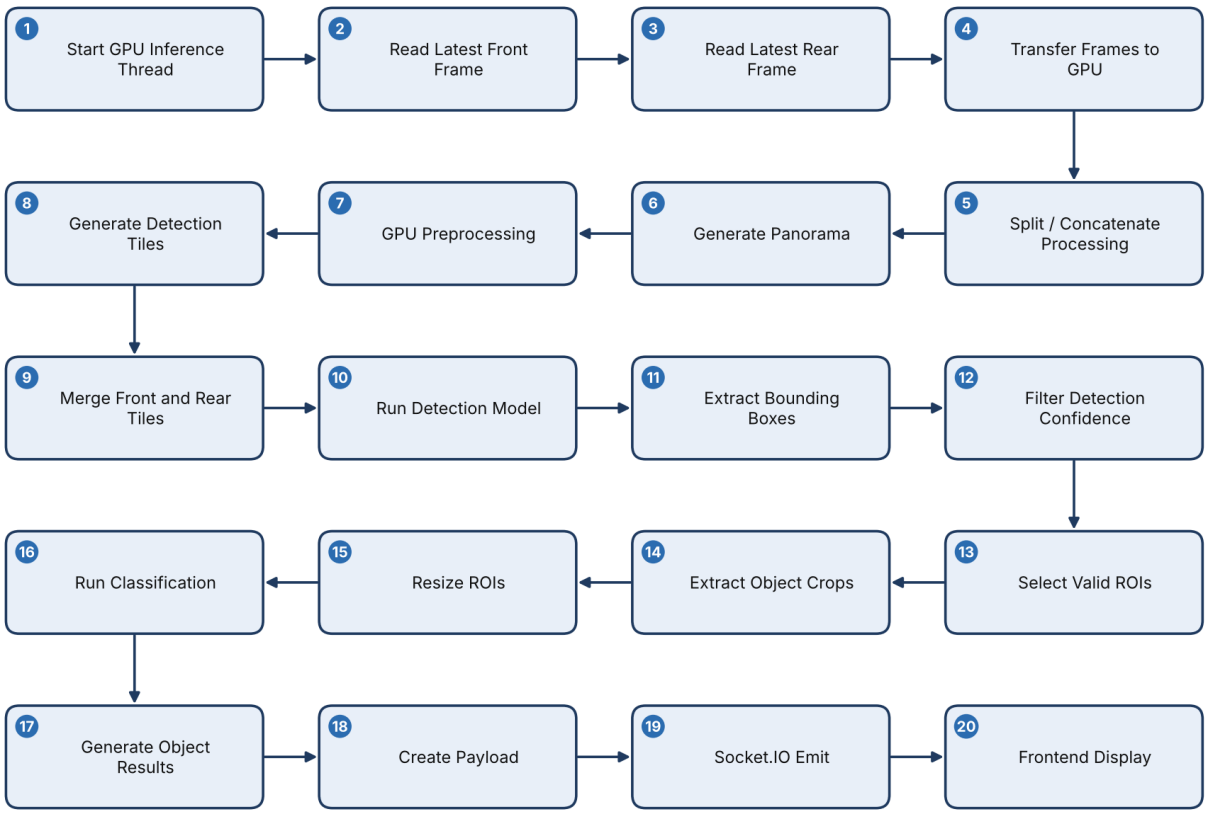


Figure 20. Complete pipeline. In parallel, the panorama is resized and JPEG-encoded, and merged into the payload. The loop repeats from step 2 while running.

24. Problems noticed in the code

These are observations from the implementation, not general recommendations.

Problem 1: Detection and classification are forcibly enabled

```
det_front_on = DETECTION_STATE["/front"]
det_rear_on  = DETECTION_STATE["/rear"]
run_detection = det_front_on or det_rear_on

cls_front_on = CLASSIFICATION_STATE["/front"]
cls_rear_on  = CLASSIFICATION_STATE["/rear"]
run_classification = cls_front_on or cls_rear_on
```

Later in the same function:

```
run_detection = True
run_classification = True
```

This overrides the previous values. Even if the frontend sets **DETECTION_STATE["/front"] = False**, the inference thread still runs detection, and classification is always enabled. To make the frontend controls work, remove the forced assignments or replace them with the intended logic.

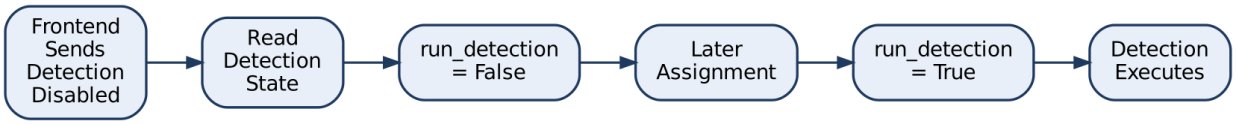


Figure 21. How the override defeats the toggle

Problem 2: Classification threshold is taken from CCD configuration

```
if sh.CAMERA_MODE == 1:
    num_classes, primaryModelOutputs = detectionModelCCD.predict(prev_tiles1)
    CLS_THRES = classificationModelCCD.CLS_THRES
elif sh.CAMERA_MODE == 0:
    num_classes, primaryModelOutputs = detectionModelIR.predict(prev_tiles11)

# later, regardless of camera mode:
CLS_THRES = classificationModelCCD.CLS_THRES
```

The later assignment runs regardless of camera mode, so the filtering threshold comes from the CCD configuration even when processing IR. Check this against your intended CCD/IR behavior.

Problem 3: Classification preprocessing runs before the toggle check

`cls_tiles1` and `cls_tiles2` are created before the classification execution logic, so preprocessing may happen even when classification is disabled. Combined with the forced `run_classification = True`, the classification toggle currently has little effect.

Problem 4: Fire and smoke variables load the same model

`model_yolo_fire` and `model_yolo_smoke` load identical weights. That may be intentional because the model has both classes, but it initializes two instances and can increase GPU memory usage. Whether two are needed depends on the implementation of `video_analytics_predict_dual_yolov26()`.

Problem 5: Several stages use synchronization

```
stream_inference1.wait_stream(stream_tile_merge)
```

This is necessary because inference must wait for the detection tiles. Still, synchronization can affect throughput:

- CUDA streams allow asynchronous execution, but dependencies still require synchronization.
- CPU operations such as **.item()** can introduce synchronization.
- GPU operations that depend on earlier streams must wait for them.

```
tile_idx = int(d_roi_tile[i])
x1 = max(0, int(x1.item()))
```

These read GPU values from Python and can force GPU-to-CPU synchronization. If this happens repeatedly for many ROIs, it can reduce performance.

25. Final summary

hw_v3.py implements a multi-threaded, GPU-accelerated video analytics pipeline:

Stage	Main component	Function
1	RTSPReader	Read video
2	PyAV	Decode frames
3	CUDA processing	Process images
4	Custom preprocessing kernels	Generate tiles
5	RF-DETR / IR detector	Detect objects
6	ROI extraction	Extract detected objects
7	Classification model	Classify objects
8	YOLOv26	Detect fire and smoke
9	X3D	Detect human conflict
10	TurboJPEG	Encode video
11	Socket.IO	Send results
12	Frontend	Display results

Recommended reading order

1. **RTSPReader**: how video enters the application.
2. **gpu_inference_thread()**: how the pipeline is coordinated (the most important function).
3. **video_analytics_fire_smoke()**: how fire and smoke analytics work.
4. **video_analytics_human_conflict()**: how temporal analytics work.
5. **resize_with_letterbox()**: how object crops are prepared.
6. **emit_worker()**: how results are transmitted.

> **Limitation:** hw_v3.py imports several custom compiled CUDA modules and external model implementations. Their internals are not visible in this file, so this document covers the code's visible behavior and the responsibilities those modules indicate.